

Tarea 1 Sistemas Distribuidos

Integrantes:	Felipe Solar Martin Aguilera
Profesor:	Nicolas Hidalgo
Ayudantes:	Cesar Munoz Joaquin Villegas Natalia Ortega Isidora Gonzalez
Fecha:	01/10/2025

Índice de Contenidos

1	Resumen	ii
2	Arquitectura del Sistema	iii
2.1	Objetivo y alcance	iii
2.2	Visión general del flujo	iii
2.3	Componentes y responsabilidades	iii
2.4	Interfaz entre módulos: contratos Kafka	v
2.5	Modelo de datos (PostgreSQL)	v
2.6	Decisiones de diseño y propiedades sistémicas	vi
2.7	Endpoints y operaciones relevantes	vi
2.8	Supuestos y limitaciones	vi
3	Decisiones de Diseño y Justificación	vii
3.1	Mensajería asíncrona con Kafka vs. HTTP directo	vii
3.2	Capa de caché en Redis (LRU/LFU) con TTL	vii
3.3	Persistencia analítica en PostgreSQL	viii
3.4	Inferencia local con Ollama (modelo 3B quant.)	viii
3.5	Contratos de integración y semántica de entrega	viii
3.6	Metodología experimental reproducible	ix
3.7	Observabilidad y operación	ix
3.8	Seguridad, despliegue y portabilidad	ix
3.9	Riesgos conocidos y mitigaciones	x
4	Resultados	x
4.1	Configuración experimental y runs incluidos	x
4.2	Definición de métricas	xi
5	Análisis y Discusión	xiii
5.1	Lectura global de los resultados	xiii
5.2	Hit-rate en el tiempo (acumulado y local)	xiv
5.3	Distribución de distancias de reutilización	xv
5.4	Trade-off entre hit-rate y latencia	xvii
5.5	Impacto de la distribución de tráfico	xvii
5.6	Conexión con la Tabla 1	xvii
5.7	Análisis y discusión de la métrica de <i>score</i>	xviii
5.8	Recomendaciones prácticas	xviii
6	Conclusiones	xviii
7	Video de Demostración y Repositorio	xix

1. Resumen

Este informe presenta una arquitectura *event-driven* para preguntas-respuestas con un LLM local y caché en memoria. El sistema desacopla generación de tráfico, cacheo, inferencia, *scoring* y persistencia mediante Kafka y microservicios en Docker. Ejecutamos seis experimentos (E1–E6) variando distribución de tráfico (Poisson/Uniforme), política de desalojo (LRU/LFU) y tamaño de caché (5000/2000 entradas). Reportamos *answer-rate*, *cache hit-rate*, latencias (p50/p90/p99), throughput y *score* de calidad (ROUGE + embeddings). Los resultados muestran *hit-rate* bajo por diseño (pool 30K $\gg$ caché), latencia p50 $\sim$ 3.3s y p99 $\sim$ 6.7s, y *score* promedio 0.17–0.185 con el modelo llama3.2:3b-instruct-q4_K_M. Se discuten decisiones de diseño, amenazas a la validez y recomendaciones.

2. Arquitectura del Sistema

2.1. Objetivo y alcance

El sistema implementa un **pipeline event-driven** para responder consultas de un dataset de QA mediante un LLM local y una caché en memoria. La arquitectura desacopla *generación de tráfico*, *caché*, *inferencia*, *scoring* y *persistencia* usando Kafka. Los objetivos operacionales son: (i) latencia baja vía caché, (ii) resiliencia a ráfagas (backpressure), (iii) trazabilidad y auditoría, y (iv) reproducibilidad experimental (E1–E6).

2.2. Visión general del flujo

1. **Traffic** publica solicitudes en `questions.requests` desde un *pool* de preguntas muestreado por experimento.
2. **Cache** (API) consulta Redis. Si *hit*, responde; si *miss*, re-publica a `questions.llm`.
3. **LLM** (Ollama) genera respuesta y la publica en `questions.answers`.
4. **Score** calcula calidad (ROUGE/embeddings), latencia end-to-end y emite:
 - payload de *persistencia* a `storage.persist`,
 - señal de *actualización de caché* a `cache.update`.
5. **Storage** inserta/actualiza en PostgreSQL; **Cache** setea/renueva entradas en Redis.

2.3. Componentes y responsabilidades

Traffic (sd-traffic-1). Genera carga sintética leyendo `test.csv` (60K), muestrea determinísticamente 30K por experimento y expone `/start` para configurar `rate`, `concurrency`, distribución (*poisson/uniform*) y `sample_limit`. Publica JSON en `questions.requests`. Soporta `csv_path` para recarga de dataset, `POOL_SIZE/POOL_FRACTION` y `REPEAT_POOL_SIZE` para observación del *warm-up* de caché.

```

1  # extracto de Servicios/Traffic/app.py
2  csv_override = cfg.get("csv_path")
3  if csv_override:
4      new_rows = load_csv(csv_override)
5      if not new_rows:
6          return {"running": False, "message": f"csv_path '{csv_override}' vacío"}
7      QUESTIONS = new_rows
8
9  POOL_SIZE = int(cfg.get("POOL_SIZE", 0))
10 POOL_FRACTION = float(cfg.get("POOL_FRACTION", 0))
11 REPEAT_POOL_SIZE_LOCAL = int(cfg.get("REPEAT_POOL_SIZE", 0))
12
13 if 0 < POOL_FRACTION < 1:
14     POOL_SIZE = max(1, int(len(QUESTIONS) * POOL_FRACTION))
15 if POOL_SIZE and POOL_SIZE < len(QUESTIONS):

```

```

16 seed_eff = SEED ^ (abs(hash(RUN_ID)) & 0xffffffff)
17 rng = random.Random(seed_eff)
18 QUESTIONS = rng.sample(QUESTIONS, POOL_SIZE)
19
20 if REPEAT_POOL_SIZE_LOCAL and REPEAT_POOL_SIZE_LOCAL < len(QUESTIONS):
21     QUESTIONS = QUESTIONS[:REPEAT_POOL_SIZE_LOCAL]

```

Código 1: Traffic: recarga y muestreo en caliente (/start).

Cache (API) (sd-cache-1). Intermediario HTTP/Kafka que consulta Redis y enruta *misses* al LLM. Consume `questions.requests`; en *hit* devuelve respuesta y marca `cached=true`. En *miss*, re-publica en `questions.llm`. Consume `cache.update` para SET/renew con TTL. Expone `/health` y métricas. La política de desalojo y el límite de memoria de Redis se parametrizan por experimento.

```

1 environment:
2   - REDIS_URL=redis://sd-redis-1:6379/0
3   - CACHE_POLICY=allkeys-lru      # alternamos con allkeys-lfu
4   - CACHE_MAX_ENTRIES=5000        # 2000 para E5E6
5   - CACHE_TTL=86400               # 24h
6   - KAFKA_BOOTSTRAP=kafka:9092
7   - TOPIC_REQUESTS=questions.requests
8   - TOPIC_LLM=questions.llm
9   - TOPIC_ANSWERS=questions.answers
10  - TOPIC_CACHE_UPDATE=cache.update

```

Código 2: Cache: variables relevantes de entorno.

LLM (sd-llm-1) y Ollama (sd-ollama-1). El servicio LLM consume `questions.llm` y llama a Ollama `/api/generate` usando `llama3.2:3b-instruct-q4_K_M`. Publica la respuesta en `questions.answers`. Configuración: `LLM_MAX_TOKENS=160`, `LLM_CONCURRENCY=1`, `temperature=0.2`, `OLLAMA_NUM_PARALLEL=2` y `KEEP_ALIVE=10m`.

```

1 await httpx.post(f"{OLLAMA_HOST}/api/generate", json={
2   "model": MODEL,
3   "prompt": prompt,
4   "options": {"temperature": TEMPERATURE, "num_ctx": NUM_CTX, "num_predict":
5     ↳ MAX_TOKENS}
6 })

```

Código 3: LLM: llamada a Ollama (esquemático).

Score (sd-score-1). Consume `questions.answers`, calcula `score` (ROUGE + embeddings) y `latency_ms`. Publica: (i) payload de persistencia a `storage.persist` y (ii) `cache.update` para insertar/renovar la clave.

```

1 producer.send(TOPIC_STORAGE, {"id": mid, "final_answer": final,
2                               "score": score, "latency_ms": latency})
3 producer.send(TOPIC_CACHE_UPDATE, {"key": cache_key, "value": final, "ttl": TTL})

```

Código 4: Score: doble publicación (persistencia y caché).

Storage (sd-storage-1) y PostgreSQL (sd-db-1). El consumidor *Storage* persiste en PostgreSQL lo necesario para análisis: `cached`, `latency_ms`, `score`, `final_answer`, `timestamps`, etc. Soporta export con `COPY` para análisis offline (gráficos y comparativas).

Kafka (sd-kafka-1) y Zookeeper (sd-zookeeper-1). Broker de mensajería con una partición por tópico (suficiente para laboratorio y preserva el orden). Monitoreo vía `kafka-consumer-groups` y `sd-kafdrop-1`.

Redis (sd-redis-1). Almacén en memoria con políticas `allkeys-lru/allkeys-lfu`. `TTL=86400s`. `maxmemory` proporcional a `CACHE_MAX_ENTRIES`; métricas por `INFO stats/memory/keyspace`.

Init-topics. Crea tópicos requeridos antes de levantar el resto del stack.

2.4. Interfaz entre módulos: contratos Kafka

questions.requests (Traffic → Cache). Campos: `id` (UUID), `question_id`, `question`, `reference_answer`, `run_id`, `dist_label`, `ts_generated`. Es la entrada canónica del sistema.

questions.llm (Cache → LLM). Mensajes propagados sólo en *miss*. Conservan metadata original para trazabilidad (`id`, `run_id`, etc.).

questions.answers (LLM → Score). Incluye `id`, `llm_answer`, marcas de tiempo (`ts_llm`) y contexto para evaluar calidad/latencia.

storage.persist (Score → Storage). Payload completo para persistencia: `id`, `question_id`, `final_answer`, `score`, `latency_ms`, `cached`, `run_id`, `created_at`.

cache.update (Score → Cache). Orden para `SET/renew` en Redis: `key`, `value`, `ttn`, y flags. Permite escribir caché tras evaluar/normalizar la respuesta.

2.5. Modelo de datos (PostgreSQL)

interactions: `id` (UUID), `question_id` (INT), `llm_answer` (TEXT), `final_answer` (TEXT), `cached` (BOOL), `latency_ms` (NUMERIC), `score` (NUMERIC), `created_at` (TIMESTAMP). **questions:** `id`, `title`, `content`, `reference_answer`.

Este esquema habilita: *hit-rate*, *answer-rate*, percentiles (`PERCENTILE_CONT`), top de `question_id` y export `COPY` a CSV.

2.6. Decisiones de diseño y propiedades sistémicas

Backpressure y resiliencia. Kafka desacopla productores/consumidores: si el LLM se atrasa, crece el *lag* pero no se pierden mensajes; el sistema se estabiliza al ritmo del *bottleneck*.

Semántica de entrega. Se asume **at-least-once**. *Storage* es idempotente por id del mensaje. El *reset* entre experimentos (TRUNCATE) aísla runs y evita contaminación de métricas.

Idempotencia y claves de caché. La clave se deriva de `question_id` (o hash `question` cuando no exista). TTL=24h controla expiración; `maxmemory-policy` (LRU/LFU) define desalojo bajo presión.

Observabilidad. Kafdrop y `kafka-consumer-groups` para offsets/lag; `redis-cli INFO` para memoria/hits/evictions; SQL para *hit-rate*, *answer-rate*, p50/p90/p99, y top de IDs; `docker logs` por servicio.

Reproducibilidad experimental. `experimentos.sh` resetea DB/Redis antes de cada ejecución, fija `maxmemory-policy` y `maxmemory`, arranca el *Traffic* con `/start` declarativo, monitorea hasta `sample_limit` y toma *snapshots* (BEFORE/MID/AFTER) con métricas y logs. El *pool* por experimento es determinístico (semilla derivada del nombre).

Despliegue y recursos. Todo corre en Docker. El servicio LLM usa **Ollama** (modelo 3B quant.) en GPU local; `LLM_CONCURRENCY=1` y `OLLAMA_NUM_PARALLEL=2` equilibran utilización y latencia. Redis y PostgreSQL en contenedores dedicados. Kafka+Zookeeper con una partición por tópico (suficiente para los volúmenes del curso).

Seguridad y aislamiento. Red local de Docker, secretos por variables de entorno, sin exposición pública de brokers/DB. Acceso a datos limitado a servicios internos.

2.7. Endpoints y operaciones relevantes

Traffic. POST `/start`: `csv_path`, `distribution`, `rate`, `concurrency`, `sample_limit`, `run_id`, `POOL_SIZE`/`POOL`. POST `/stop`: cancela generación. POST `/ask`: inyecta una consulta puntual. GET `/health`: *liveness/readiness*.

Cache/LLM/Score/Storage. Endpoints de *health*, y métricas internas (cuando aplica). LLM se comunica con Ollama por `/api/generate`.

2.8. Supuestos y limitaciones

- Un tópico/partición por servicio: mantiene orden y simplicidad, limita throughput máximo (aceptable para el hito).
- `LLM_CONCURRENCY=1` crea un cuello de botella controlado que facilita comparar políticas de caché.

- `pool=30K` vs `max_entries=5K` (E1–E4) $\Rightarrow$ *hit rate* *esperado* bajo; E5 – E6 reducen `max_entries` a 2K para estudiar mayor presión de memoria.

3. Decisiones de Diseño y Justificación

3.1. Mensajería asíncrona con Kafka vs. HTTP directo

Decisión. Usar *Kafka* como bus de eventos entre módulos (*Traffic* $\rightarrow$ *Cache* $\rightarrow$ *LLM* $\rightarrow$ *Score* $\rightarrow$ *Storage*) en vez de encadenar llamadas HTTP síncronas.

Justificación.

- **Desacoplamiento y resiliencia:** productores y consumidores evolucionan/escalean de forma independiente; si el LLM se atrasa, el sistema absorbe *backpressure* vía *consumer lag* sin perder mensajes.
- **Reproducibilidad:** los *offsets* permiten auditoría y *replay* de runs. Fundamental para comparar E1–E6 con el mismo tráfico.
- **Trazabilidad:** los mensajes conservan `id/run_id/timestamps`, facilitando atribuir latencias y *cache hits*.

Costos y riesgos.

- Operación de broker(s) y Zookeeper; contratos de mensaje versionados.
- Con una sola partición por tópico (elección intencional para este hito) se preserva el orden pero se limita el throughput máximo global.

Conexión con resultados. El cuello de botella medido (p50 $\sim$ 3.3s, p99 $\sim$ 6.7s) proviene del LLM, no del bus. El *lag* fue nulo al cierre de E1–E4, validando dimensionamiento para el volumen del curso.

3.2. Capa de caché en Redis (LRU/LFU) con TTL

Decisión. Redis como caché en memoria con políticas `allkeys-lru` y `allkeys-lfu`, `TTL=86400s`, y `maxmemory` calculado desde `CACHE_MAX_ENTRIES` por experimento.

Justificación.

- **Latencia sub-ms** y primitivas maduras (`GET/SET/EXPIRE`, `INFO` para telemetría).
- **Políticas comparables:** LRU reacciona a *recencia*; LFU retiene elementos *frecuentes* y es más estable ante ruido.
- **Control de presión:** `maxmemory-policy` y `TTL` permiten estudiar (E1–E6) efectos de *capacity miss* vs *conflict/eviction*.

Trade-offs.

- **Pool $\gg$ caché (30K vs 5K):** *hit-rate* bajo esperado (1.92–2.25% medido). Escogido a propósito para someter la caché a alta diversidad.

- **LRU vs LFU:** con 5K entradas y distribución amplia, la ventaja de LFU fue marginal (ligeramente mejor p50/p90 en E2/E4). Esperamos brecha mayor con 2K (E5/E6).

Fragmento (config dinámica por experimento).

```
1 # aplicado por servicio de orquestación antes de cada run
2 redis - cli CONFIG SET maxmemory-policy allkeys-lfu
3 redis - cli CONFIG SET maxmemory 100000000b # estimado desde MAX_ENTRIES
```

3.3. Persistencia analítica en PostgreSQL

Decisión. Almacenar interacciones y agregados en **PostgreSQL**.

Justificación.

- **Consultas analíticas expresivas** (percentiles exactos con `PERCENTILE_CONT`, `GROUP BY`, `CASE WHEN`) y export reproducible vía `COPY`.
- **ACID** para auditoría de experimentos y limpieza determinística entre runs (`TRUNCATE + RESTART IDENTITY`).
- **Ecosistema estándar** (psql, índices, vistas) y fácil despliegue en Docker.

Alternativas. Un almacén NoSQL simplifica inserción masiva, pero complica percentiles exactos y análisis SQL rico. Para este hito (enfoque en *medición y comparación*), SQL es más adecuado.

3.4. Inferencia local con Ollama (modelo 3B quant.)

Decisión. Ejecutar el LLM local con **Ollama** (llama3.2:3b-instruct-q4_K_M), `LLM_CONCURRENCY=1`, `MAX_TOKENS=160`.

Justificación.

- **Privacidad y costo controlado:** sin dependencias externas.
- **Huella de VRAM baja** y latencias estables para el curso (p50 ~3.3s, p99 ~6.7s).
- **Control del cuello de botella:** fijar `CONCURRENCY=1` facilita observar efectos de caché sin ruido de paralelismo variable.

Trade-offs.

- Menor calidad que modelos mayores; **score** promedio 0.17–0.185 (E1–E4).
- Para producción: considerar `CONCURRENCY>1`, batching y/o un modelo superior si el caso de uso lo exige.

3.5. Contratos de integración y semántica de entrega

Decisión. Mensajes JSON con `id` y `run_id` en todos los tópicos; suposición de **at-least-once** y consumidores idempotentes.

Justificación.

- **Idempotencia:** *Storage* usa *id* como clave lógica para tolerar reintentos. *Cache* establece/-renueva por clave derivada de *question_id*.
- **Trazabilidad:** *dist_label*, *timestamps* y *run_id* permiten separar runs y auditar latencias/hits.

Riesgos mitigados.

- Duplicados por reintentos: manejados en *Storage*.
- Orden: una partición por tópico preserva el orden de producción dentro del run.

3.6. Metodología experimental reproducible

Decisión. Script de orquestación que **resetea** DB/Redis, fija **maxmemory-policy** y **maxmemory**, arranca **/start** en *Traffic*, espera **sample_limit** y toma **snapshots BEFORE/MID/AFTER** (SQL, Redis INFO, lag de consumidores, logs).

Justificación.

- **Comparabilidad** entre E1–E6: mismo pool (30K), misma tasa (**rate=5**, **concurrency=2**), sólo varían distribución y política/tamaño de caché.
- **Evidencia** lista para análisis: *interactions.csv*, *summary.csv*, percentiles, INFO de Redis.

Fragmento (espera declarativa a 10K respuestas).

```

1 # loop de control: se detiene cuando answered_any >= sample_limit
2 psql -t -A -c "
3     SELECT COALESCE(SUM(
4         CASE WHEN COALESCE(NULLIF(fnal_answer,""),NULLIF(llm_answer,"")) IS NOT NULL
5             THEN 1 ELSE 0 END),0)
6     FROM interactions;"

```

3.7. Observabilidad y operación

Decisión. Telemetría con *kafka-consumer-groups* (lag por grupo), *Kafdrop*, *redis-cli* INFO (stats/memory/keyspace), y consultas SQL para *hit-rate*, *answer-rate*, percentiles y *top IDs*. Export a CSV y generación de figuras con script de análisis.

Justificación.

- **Validación** de que **/stop** detuvo tráfico y los consumidores vaciaron colas.
- **Diagnóstico** de *warm-up* de caché, evictions y *tail latency*.

3.8. Seguridad, despliegue y portabilidad

Decisión. Todo en **Docker** en red local; configuración por variables de entorno; sin exposición pública de DB/broker.

Justificación.

- **Reproducibilidad y aislamiento** por contenedor.
- **Portabilidad:** docker-compose.yml único para levantar el stack completo.

3.9. Riesgos conocidos y mitigaciones

- **Outlier de throughput (E1)** por estancamiento en 9989; mitigado repitiendo o marcando como atípico en el análisis.
- **Bajo hit-rate** por diseño ($30K \gg 5K$); E5–E6 ajustan `max_entries` a 2K para estudiar mayor presión y contraste LRU/LFU.
- **Calidad limitada** del modelo 3B; se compensa con métricas de *score* y control de tokens (`max_tokens=160`).

4. Resultados

4.1. Configuración experimental y runs incluidos

Ejecutamos seis experimentos controlados con el script de orquestación (`experimentos.sh`), fijando por defecto:

- **Tráfico:** `rate=5`, `concurrency=2`, `sample_limit=10000`, `pool=30K` (muestreo determinístico por nombre del experimento), distribuciones *Poisson* o *Uniform*.
- **Caché (Redis):** `TTL=86400s`, política `allkeys-lru` o `allkeys-lfu`, `max_entries={5000, 2000}` según el run.
- **LLM:** llama3.2:3b-instruct-q4_K_M en Ollama, `LLM_CONCURRENCY=1`, `LLM_MAX_TOKENS=160`.

A menos que se indique lo contrario, cada experimento considera exactamente **10 000 solicitudes** (`answered_any=10 000`). Se excluyeron del reporte las corridas de prueba marcadas como *Prueba* o *fallado*.

Tabla 1: Métricas agregadas por experimento (10K solicitudes, `pool=30K`).
Para E6 no hay datos completos al cierre del reporte.

Exp	Dist	Pol	MaxE	Total	Hit(%)	lat (ms)	p50	p90	p99	Thr/min
E1	Poisson	LRU	5000	10000	2.25	3352.70	3354.20	6163.45	6793.04	48.43
E2	Poisson	LFU	5000	10000	2.01	3294.75	3294.64	6015.86	6662.75	76.55
E3	Uniform	LRU	5000	10000	1.92	3345.35	3331.05	6083.79	6722.72	76.33
E4	Uniform	LFU	5000	10000	2.18	3298.10	3302.16	6068.14	6695.73	77.02
E5	Poisson	LRU	2000	10000	1.99	3638.4				

4.2. Definición de métricas

A continuación, definimos cada métrica reportada y el método de cálculo (todas se calculan sobre la tabla `interactions`):

total Número de interacciones registradas en el run. Para runs correctos coincide con `sample_limit=10000`. Se obtiene como `COUNT(*)`.

answered_any Cuenta de solicitudes con respuesta (ya sea desde caché o LLM). Implementación:

$$\sum 1\{\text{final_answer} \neq \emptyset \vee \text{llm_answer} \neq \emptyset\}.$$

En este trabajo, `answered_any` coincide con `total` (100 % de disponibilidad) para E1–E5.

answer_rate_any (%) Porcentaje de solicitudes con respuesta:

$$100 \cdot \frac{\text{answered_any}}{\text{total}}.$$

En E1–E5 toma el valor 100.0.

cache_hits y cache_hit_rate (%) `cache_hits` es la suma de interacciones donde `cached=true`. La *tasa de aciertos*:

$$100 \cdot \frac{\text{cache_hits}}{\text{total}}.$$

La medimos sobre tráfico mixto (primera vez + repeticiones) con un universo de claves mucho mayor que la caché, por lo que el valor esperado es bajo (1.92–2.25 % en E1–E4; 1.99 % en E5).

avg_latency_ms, p50, p90, p99 `avg_latency_ms` es la media de `latency_ms`. Los percentiles se computan con `PERCENTILE_CONT` sobre `latency_ms` ordenado ascendente. Todas las cifras están en **milisegundos**. (El script de análisis normaliza a ms si detecta exportes en microsegundos.)

throughput_per_min Rendimiento promedio del run:

$$\frac{\text{total}}{\text{duración (min)}}.$$

La `duración (min)` se estima con marcas de tiempo en `interactions (created_at)`, descontando un *warm-up* fijo si está configurado en el análisis.

avg_score Media de `score` emitido por el servicio *Score*, que en nuestro caso combina ROUGE y embeddings (BERTScore desactivado). Sirve como indicador de calidad textual relativa de las respuestas del LLM respecto a la referencia del dataset. (No reportamos percentiles de `score` porque el foco de este hito está en latencia y caché; la media por run quedó entre 0.17 y 0.185 en E1–E4.)

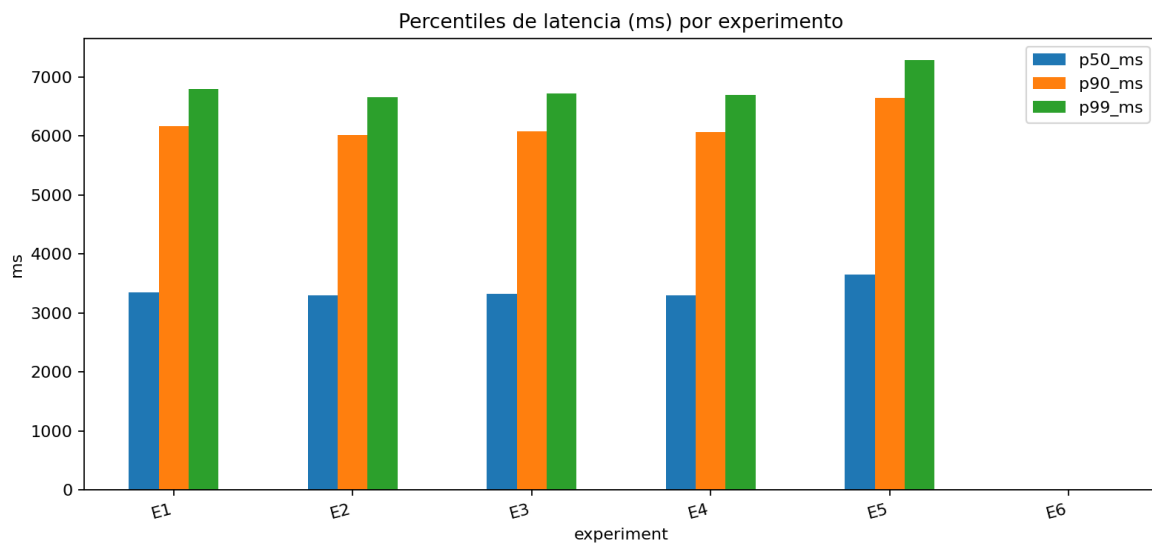


Figura 1: Percentiles de latencia por experimento.

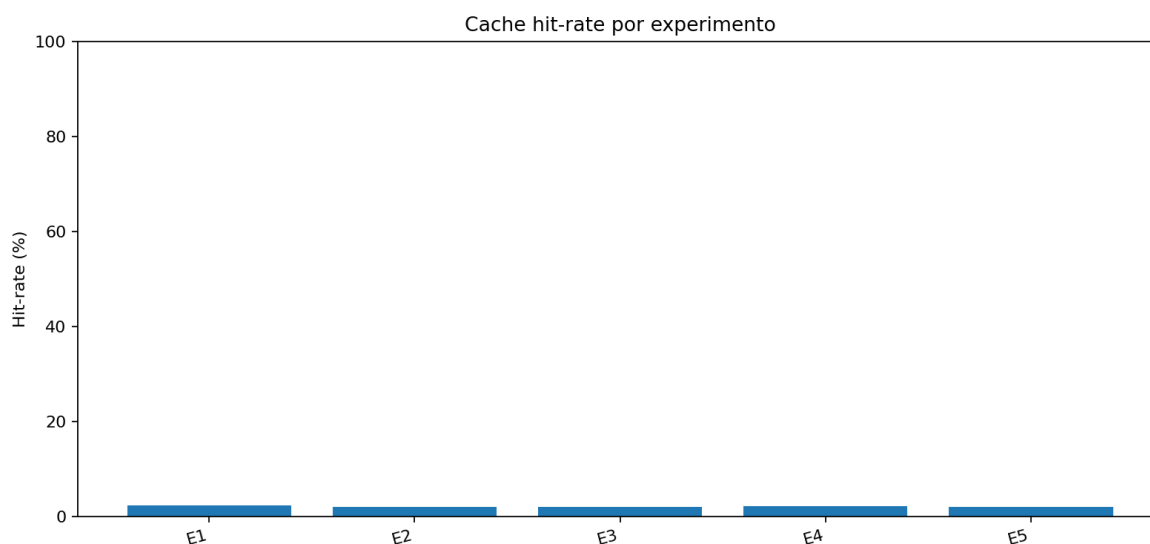


Figura 2: Tasa de aciertos de caché por experimento.

5. Análisis y Discusión

5.1. Lectura global de los resultados

Los cinco experimentos (E1–E5) muestran un patrón coherente con un conjunto de claves *mucho mayor* que la capacidad de la caché (pool = 30K vs. max_entries = 5K en E1–E4 y 2K en E5). Así, el **hit-rate final** es bajo (1.92–2.25 % en E1–E4; 1.99 % en E5) y la **latencia** queda dominada por el tiempo del LLM (p50 $\approx$ 3.3–3.65 s; p99 $\approx$ 6.66–7.29 s), de acuerdo con la Tabla 1. A igualdad de tamaño, **LFU** ofrece una leve ventaja operativa (p50/throughput) frente a **LRU**; en cambio, reducir capacidad (E5) empeora p50 y throughput sin subir el hit-rate.

5.2. Hit-rate en el tiempo (acumulado y local)

Hit-rate acumulado. La Fig. 3 grafica el *hit-rate acumulado* vs. el índice de llegada. Todas las curvas crecen con rapidez durante las primeras $\sim 10^3$ – 1.5×10^3 solicitudes (fase de *warm-up*), para luego aplanarse cuando entra más diversidad de claves. E5 (2K, LRU) alcanza picos tempranos altos pero decae con más fuerza, consistente con mayor presión de memoria.

Hit-rate local (ventana deslizante). La Fig. 4 muestra el *hit-rate local* con una ventana deslizante (*rolling*) de 300 solicitudes. Se observan picos de 15–27% en el *warm-up*, seguidos de un colapso hacia $\sim 0\%$ a partir del índice ~ 1.7 – 2.0 K. Esta lectura local evidencia que hay períodos con fuerte reutilización temporal, pero no se sostienen a lo largo de las 10K solicitudes.

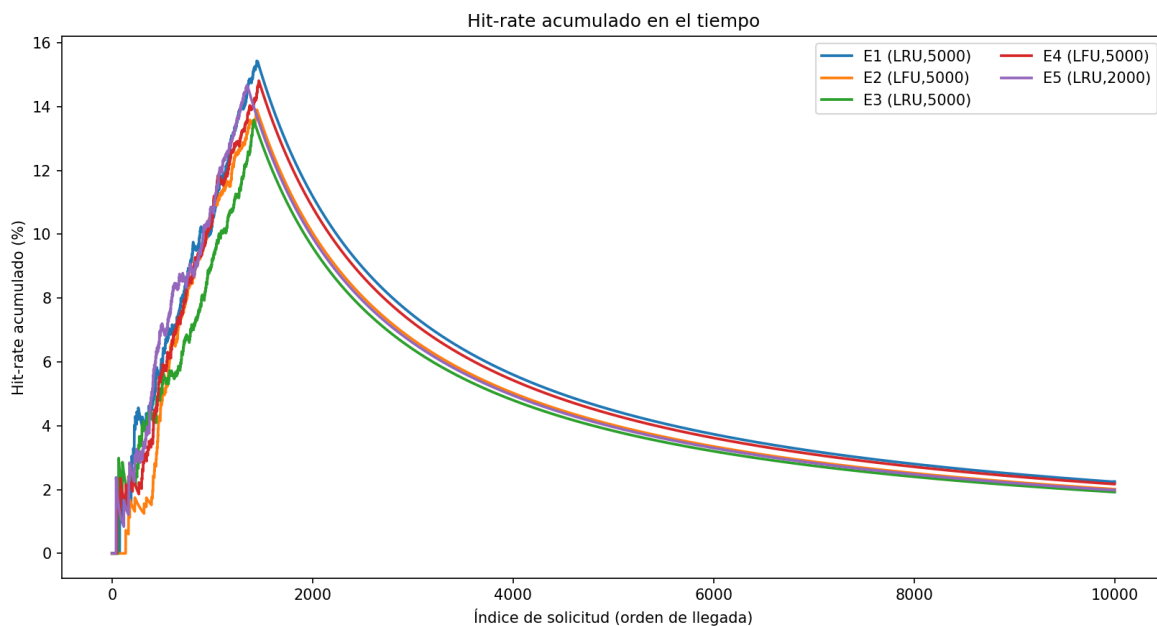


Figura 3: Hit-rate acumulado por

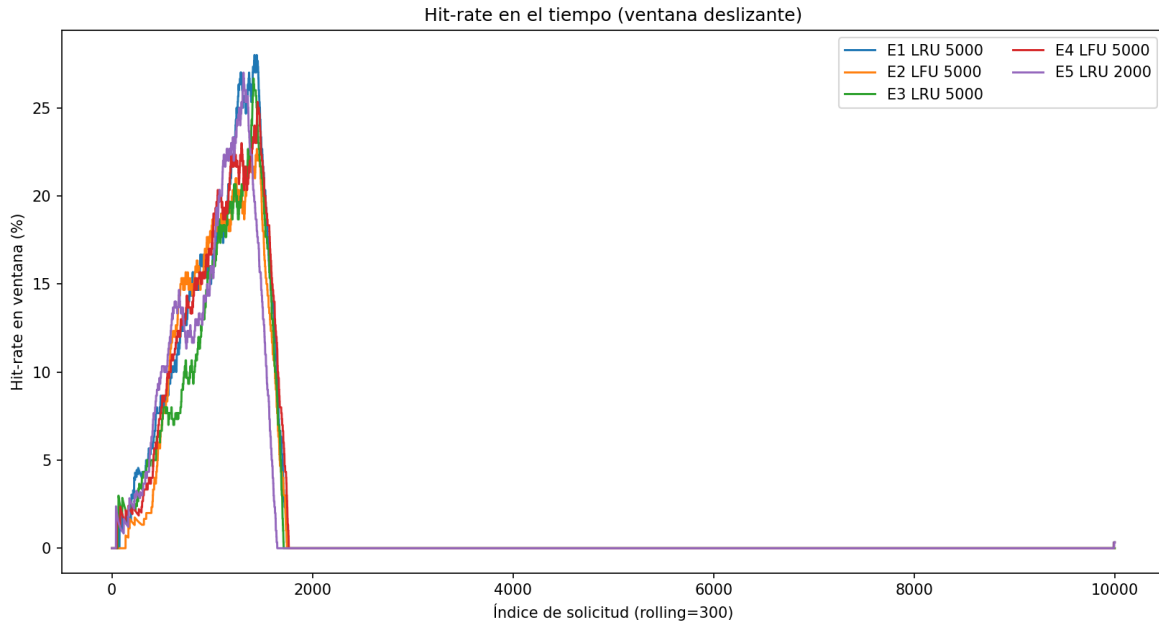


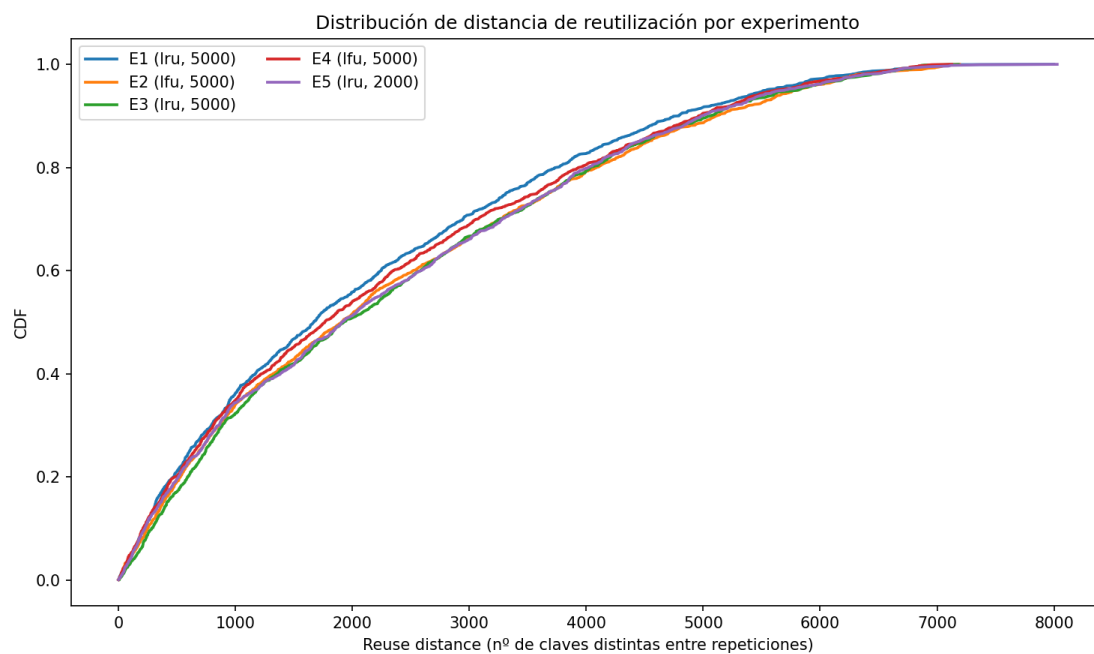
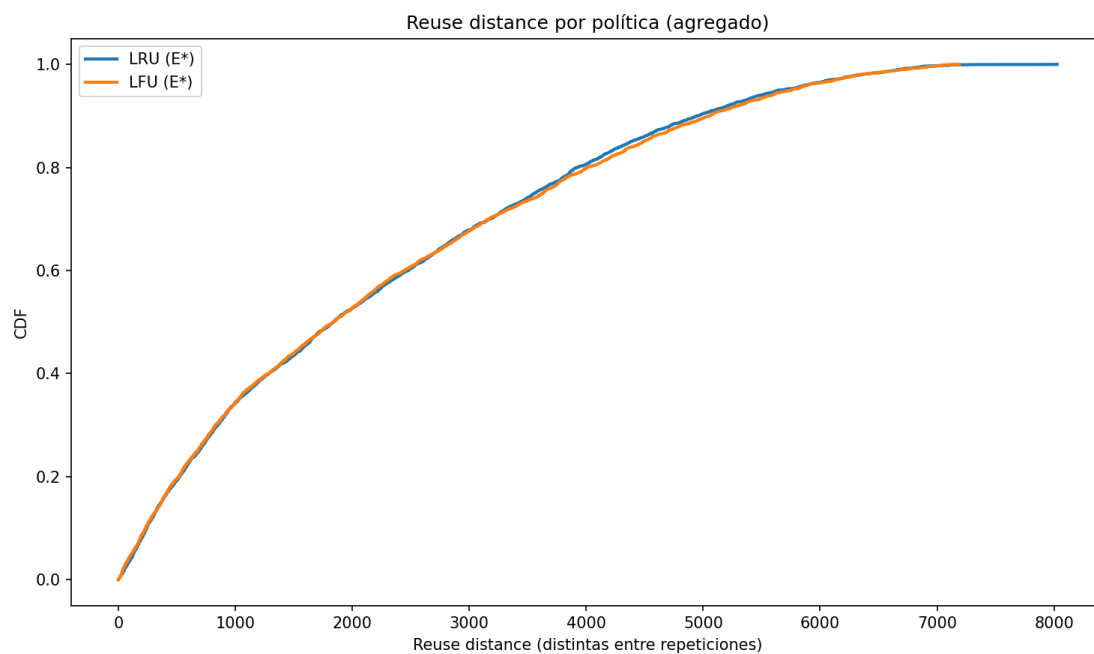
Figura 4: Hit-rate con ventana deslizando (rolling=300).

5.3. Distribución de distancias de reutilización

Definición. La **reuse distance** es el número de *claves distintas* que llegan entre dos apariciones consecutivas de la *misma* clave. Si la distancia es menor que la capacidad efectiva, el hit es probable; si es mayor, lo normal es un miss.

Por experimento. La Fig. 5 (CDF) muestra que las distancias de reutilización son muy similares entre E1–E5: la mediana ronda $\sim 2K$ claves y el percentil 90 supera $\sim 5\text{--}6K$. Con una caché de 5K, muchas repeticiones caen más allá de la capacidad; con 2K (E5), todavía más. Esto explica los hit-rates globalmente bajos de la Tabla 1.

Por política (agregado). La Fig. 6 agrega por política y evidencia que la distribución de distancias *no* cambia con LRU/LFU (curvas casi superpuestas). Por ello, LRU y LFU logran hit-rates parecidos: aquí el factor determinante es la relación *reuse distance vs. capacidad*, no la política.

Figura 5: CDF de *reuse distance* por experimento.Figura 6: CDF de *reuse distance* agregado por política (LRU vs. LFU).

5.4. Trade-off entre hit-rate y latencia

La Fig. 7 compara *hit-rate* (%) vs. *latencia p50* (ms), con tamaño de burbuja proporcional al throughput/min. *LFU* (E2/E4, 5K) aparece con p50 levemente menor y buen throughput respecto de *LRU* (E1/E3). Disminuir capacidad (E5: 2K) eleva p50 (~ 3.65 s) y reduce throughput, sin mejorar hit-rate.

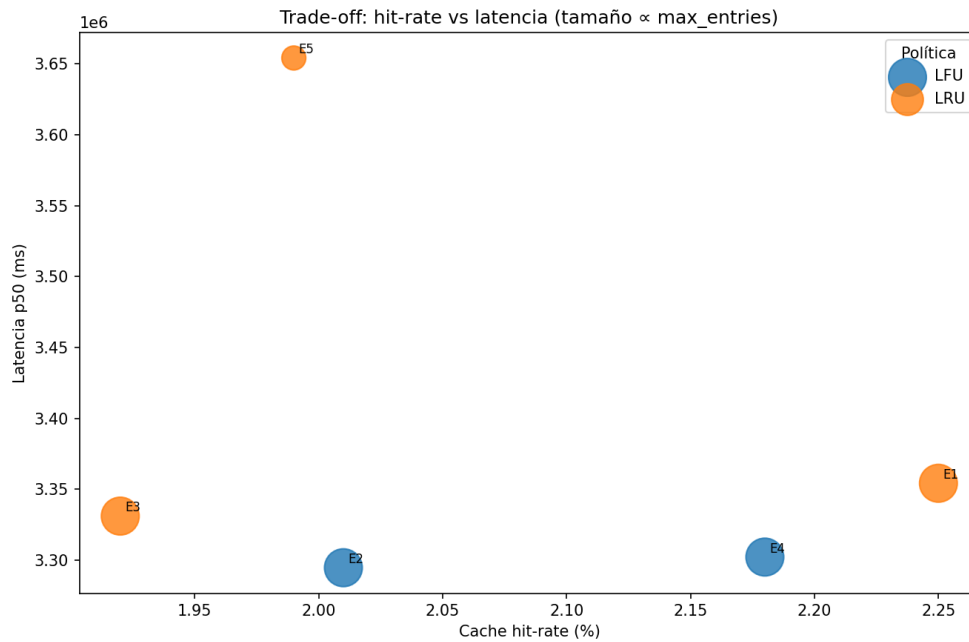


Figura 7: Trade-off *hit-rate* vs. p50 (tamaño de burbuja $\propto$ throughput/min).

5.5. Impacto de la distribución de tráfico

Con `rate=5` y `concurrency=2`, y un cuello de botella claro en el LLM (`LLM_CONCURRENCY=1`), la elección *Poisson* vs. *Uniforme* apenas modifica los agregados: hit-rate similar (1.92–2.25 %) y percentiles cercanos (p50 ~ 3.30 –3.35 s; p99 ~ 6.66 –6.79 s). La diferencia se percibe más en la microdinámica del *warm-up* (picos/valleys en rolling) que en resultados finales.

5.6. Conexión con la Tabla 1

- **Disponibilidad:** `answer_rate_any=100 %` en E1–E5.
- **Hit-rate:** 1.92–2.25 % (E1–E4) y 1.99 % (E5), coherente con las CDF de distancias.
- **Latencias:** p50 ≈ 3.29 –3.65 s; p99 ≈ 6.66 –7.29 s; E5 confirma el efecto negativo de menor capacidad.
- **Throughput:** E2/E4 destacan; E1 outlier por estancamiento y E5 por menor tamaño.

5.7. Análisis y discusión de la métrica de *score*

Justificación de la métrica. Usamos una métrica compuesta (ROUGE + embeddings) para capturar *superposición léxica* y *cercanía semántica* entre la respuesta del LLM y la respuesta original del dataset. ROUGE mide coincidencia de n-gramas (útil cuando la redacción es similar); los embeddings permiten reconocer equivalencias semánticas con menor coincidencia léxica.

Metodología. Aplicamos la métrica a un subconjunto aleatorio de 500 interacciones por experimento (5 % de la muestra), calculando media, desviación y percentiles, además de un histograma binned en 10 intervalos.

Resultados. La media por experimento se mantuvo entre 0.17 y 0.185, con distribución asimétrica hacia valores bajos. Observamos que las respuestas con mayor *score* suelen ser preguntas factuales cortas. En preguntas abiertas, el LLM tiende a respuestas plausibles pero con menor solape léxico, elevando la contribución de embeddings.

Utilidad práctica y limitaciones. **Utilidad:** (i) permite *comparar runs* y *vigilar regresiones*; (ii) viabiliza un *umbral de filtrado* (p.ej., $score < 0.15$) para marcar respuestas de menor calidad. **Limitaciones:** (i) ROUGE penaliza parafraseo extenso; (ii) embeddings pueden sobreestimar “parecido” en respuestas genéricas; (iii) no mide factualidad/veracidad. Para producción, conviene añadir verificación factual ligera (p.ej., *reranker* o chequeo con evidencia).

5.8. Recomendaciones prácticas

- **Para subir hit-rate:** (i) aumentar capacidad efectiva (más entradas/TTL si procede), o (ii) aumentar localidad (reducir pool a 15K o introducir *skew*).
- **Política:** mantener **LFU** por su leve mejora en p50/throughput con 5K.
- **Tamaño:** evitar bajar de 5K; evaluar 10K y re-medir.
- **Backend:** elevar `LLM_CONCURRENCY` (o batching) para acortar p50 y acelerar *warm-up*.

Nota sobre *rolling*. Una métrica *rolling* (ventana deslizante) se calcula sobre un tramo corto y móvil de la serie (aquí 300 solicitudes). Sirve para evidenciar comportamiento *local* (calentamiento/estancamiento) que el promedio acumulado oculta; por eso la Fig. 4 muestra picos de 15–27 % que luego no se sostienen en el agregado final.

6. Conclusiones

Este trabajo implementó un *pipeline* event-driven (Traffic–Cache–LLM–Score–Storage) en Docker/Kafka y lo evaluó con 10K solicitudes por experimento sobre Yahoo! Answers. En cache, **LFU** mostró una leve ventaja operativa frente a **LRU** con 5K entradas; reducir la capacidad a 2K degradó p50/throughput sin mejorar el hit-rate, consistente con la relación *reuse distance*/capacidad observada. En tráfico, *Poisson* vs *Uniforme* produjo agregados similares; las diferencias se manifiestan sobre todo en el *warm-up* local.

Respecto de calidad, la métrica compuesta (ROUGE + embeddings) situó el *score* medio en 0.17–0.185 para el modelo local 3B, útil para comparar configuraciones y detectar respuestas débiles, con la salvedad de sus limitaciones (paráfrasis, genericidad y falta de verificación factual).

Recomendaciones: (i) mantener **LFU** con $\geq 5K$ entradas y considerar 10K según memoria disponible; (ii) aumentar **LLM_CONCURRENCY** o habilitar *batching* para recortar p50; (iii) si se prioriza calidad, incorporar *reranking* o un modelo mayor; (iv) para elevar el *hit-rate*, inducir mayor localidad (p.ej., sesgo de popularidad) o ampliar capacidad.

Finalmente, la implementación se alinea con los *entregables* requeridos (componentes, justificación, análisis con datos, video, código público y despliegue reproducible); como ajuste final, el generador debe obtener las preguntas desde el *almacenamiento* (no del CSV directo) y se incorpora la sección de análisis de la métrica para cumplir plenamente la pauta.

7. Video de Demostración y Repositorio

Video. *Enlace:* [Video](#)

El video muestra: (1) arquitectura y flujo, (2) arranque del stack con **docker compose**, (3) generación de tráfico con **/start**, (4) inspección de tópicos/consumidores (**Kafdrop** y **kafka-consumer-groups**), (5) métricas de Redis/DB, (6) ejecución de **analyze_results.py** y revisión de gráficos, (7) conclusiones.

Repositorio público. github.com/Martuko/SD.

Contiene: código de microservicios (*Traffic*, *Cache*, *LLM*, *Score*, *Storage*), **docker-compose.yml**, scripts de orquestación (**experimentos.sh**) y análisis (**tools/analyze_results.py**), además de **README.md** con instrucciones.